

Datalake Foundation

Documentation

Generated on February 19, 2026

A metadata-driven data ingestion library
for Apache Spark and Delta Lake.

Bronze to Silver layer processing
with pluggable strategies.

Table of Contents

1. Home
2. Release Notes
3. Entity Configuration
4. Metadata Sources
5. Expression Languages
6. Log Level Configuration
7. Processing Strategies
8. Streaming Processing
9. Watermarks
10. Delete Inference
11. IO Output Modes
12. Microsoft Fabric Compatibility

Home

1. Introduction and scope

Datalake Foundation is an open-source data ingestion library developed around **Data Lakehouse** principles. It acts as a *foundation layer* between the raw **bronze** storage area and the curated **silver** layer. The library reads *data slices* (Parquet files) produced by upstream ingestion pipelines, applies metadata-driven transformations and merges them into Delta Lake tables. Its primary objective is to standardise ingestion logic across projects by supplying pre-built processing strategies, metadata handlers and utility routines.

The documentation below explains how to use the library in your environment, describes the main processing strategies and summarises the configurable options. It assumes familiarity with Apache Spark, Delta Lake and Scala-based Databricks environments. Installation guidance is kept deliberately brief; adapt to your organisation's packaging standards.

2. Prerequisites and installation

The library targets Databricks Runtime **17.3 LTS** or later and is compiled for **Scala 2.13.16**. It depends on **Spark 4.0.0** and **Delta Lake 4.0.0**. Ensure the following before deploying:

- A Spark cluster (Databricks or vanilla) with Scala 2.13 support and Delta Lake connectors.
- Access to the Bronze and Silver storage layers using a unified path convention (see section 5).
- A metadata repository in either JSON files or a SQL Server function (see section 4).
- Standard build tooling such as *sbt* to compile the Scala sources into a JAR.

Quick installation steps

1. **Obtain the source** – clone the repository or download the release package.
2. **Build the library** – run `sbt package` to compile the Scala code and produce a JAR. alternatively you can download the libraries from the [github page](#).
3. **Deploy to your cluster** – upload the JAR to your Spark/Databricks cluster or include it as a library in your job configuration.
4. **Verify dependencies** – make sure the cluster has the right Spark & Delta versions and that JSON/SQL metadata is reachable.

Note: If you have a custom packaging workflow (e.g., Databricks wheel or AWS EMR bootstrap action), adapt the steps accordingly. The library is platform-agnostic as long as the above conditions are met.

3. Processing strategies

At the heart of Datalake Foundation is the `Processing` class (in `datalake.processing`). You instantiate it with an `Entity` (from metadata) and a `slice` file name. You then call `Process`. The process strategy is automatically chosen (set in metadata of the entity). The examples below are illustrative, normally you would call `processing.Process()` (without parameter) the parameter forces the strategy.

3.1. Full load

Full is used for initial loads or full refreshes. It performs an **overwrite** write into the Silver table (dynamic partition mode).

```
val entity = metadata.getEntity(42)
val processing = new Processing(entity, "2025-07-01-slice.parquet")
processing.Process(Full)
```

3.2. Delta (merge) processing

Merge supports incremental (delta) processing and executes a Delta Lake **MERGE** into the existing Silver table. Typical behaviour:

- Falls back to a full load when the Silver table does not exist.
- Optionally filters on partition values present in the slice to limit the merge scope.
- Logs warnings on schema drifts.
- Applies upsert logic based on a computed primary key and a source hash:
 - Marked-as-deleted source rows update the target to deleted.
 - Different hashes update all columns.
 - Identical rows only update **lastSeen**.
 - No match → insert.

3.3. Historic (SCD Type 2) processing

Historic implements SCD Type 2. It keeps previous versions and tracks validity using **ValidFrom**, **ValidTo**, **IsCurrent** and **SourceHash**:

- First run defaults to a full load.
- Subsequent runs match on primary key where **IsCurrent = true**.
- Changed rows are closed off (**ValidTo** set, **IsCurrent = false**) and a new row is inserted.
- New versions are appended to the Silver table.

3.4. Streaming processing

Stream enables continuous micro-batch ingestion using Spark Structured Streaming. It supports file-based sources (Parquet, JSON, CSV, ORC, Avro), message queues (Kafka, Kinesis) and a test source (rate). Each micro-batch goes through the same transformation pipeline as batch strategies, then writes using a configurable write strategy (full, merge or historic).

```
val entity = metadata.getEntity(42)
val processing = new Processing(entity, "") // streaming reads from its configured source, not a slice file
val query: Option[StreamingQuery] = processing.Process(Stream)

// StreamingQuery lifecycle
query.foreach { q =>
  q.awaitTermination() // blocks until stopped or error
  q.stop()              // graceful shutdown
}
```

Streaming entities return **Some(StreamingQuery)** instead of **None**. The caller is responsible for managing the query lifecycle.

3.5. Processing time override

All strategies accept an optional `processing.time` option (ISO8601). If absent, current time is used. Invalid timestamps are logged and default to current time.

```
val options = Map("processing.time" -> "2025-05-05T12:00:00")
val processing = new Processing(entity, "2025-04-30-slice.parquet", options)
processing.Process(Historic)
```

Disclaimer: The library does not validate temporal succession; providing an earlier `processing.time` than already processed data can result in inconsistent `ValidFrom/ValidTo` ranges.

4. Metadata management

4.1. JSON metadata

Use `JsonMetadataSettings` to load metadata from a JSON file. The JSON defines:

- **Connections** – e.g., storage accounts or database instances.
- **Entities** – id, source connection, destination table, column definitions (with types & business key indicators), optional watermark columns and preferred processing type (Full, Merge, Historic).
- **Environment** – global settings such as timezone and system field prefixes.

Duplicate entity identifiers are rejected. Use `getEntity(id)` to retrieve an `Entity` object for processing.

4.2. JSON folder metadata

Use `JsonFolderMetadataSettings` to load metadata from multiple JSON files in a directory. Files are merged together, allowing modular organization of connections and entities.

4.3. SQL Server metadata

Use `SqlMetadataSettings` to read JSON configuration from SQL Server via JDBC (e.g., function `cfg.fnGetFoundationConfig()`). Provide a `SqlServerSettings` case class (server, port, database, username, password). Errors are logged and raised.

4.4. Metadata-driven helpers

- `getConnectionEntities(connection)` – all entities for a connection.
- `getGroupEntities(group)` – all entities in a logical group.
- `getConnection(...)` / `getConnectionByName(...)` – fetch connection definitions.
- `getEnvironment()` – retrieve environment settings.

These helpers enable dynamic pipeline generation (e.g., loop all entities of a connection).

5. Folder structure

Assumed storage layout:

Bronze layer – raw Parquet slices

```
<root_folder>/bronze/<connection>/<entity_name>/<slice_file>
```

Silver layer – processed Delta tables

```
<root_folder>/silver/<connection>/<entity_name>/<slice_file>
```

The **Entity** metadata exposes these paths via `entity.getPaths`. **Processing** composes the full slice path using `paths.bronzepath` + the slice file name.

6. Transformations and system columns

Before writing to Silver, the library applies:

- **Inject transformations** – custom logic declared in metadata.
- **Calculated columns** – derived fields.
- **Source hash** – SHA256 over all columns to detect changes.
- **Temporal tracking columns** – `ValidFrom`, `ValidTo`, `IsCurrent` for historic processing.
- **Primary key generation** – builds a hash from the business key columns if not present; rejects duplicates.
- **Column casting, renaming and normalisation** – schema enforcement.
- **Deleted flag** – to physically/logically mark deletions during merge.
- **Last seen** – timestamp updates for re-processed records.

7. Data Factory item generator

Provides a structured list (JSON/array) of connections, entities and slice files for use in orchestration tools like Azure Data Factory:

1. Call the generator with metadata settings and an optional date range.
2. Receive a structured set of entities and slice identifiers.
3. Feed the list into a For Each activity to execute notebooks/scripts that run **Processing** for each item.

8. Migration to Scala 2.13

Datalake Foundation is upgraded to Scala 2.13 for Databricks Runtime 17.3 LTS. Ensure:

- Your custom libraries are also built for Scala 2.13.
- Spark and Delta versions on your cluster match the requirements.
- You rebuild JARs that were compiled for Scala 2.12.

9. Summary

IO Output

Use `output_method` in the `environment` section to configure whether entities return file system paths (`paths`) or catalog table names (`catalog`). Individual entities can override this setting in their `settings` block. The default value is `paths` for backward compatibility. See [IO Output Modes](#) for detailed configuration options.

Documentation

For detailed documentation on specific topics, see:

Configuration

- [Entity Configuration](#) – Entity structure, column definitions, field roles, and settings hierarchy
- [Metadata Sources](#) – JSON, SQL Server, and folder-based metadata configuration

Processing

- [Processing Strategies](#) – Full, Merge, and Historic strategies in depth
- [Streaming Processing](#) – Continuous micro-batch ingestion with Structured Streaming
- [Watermarks](#) – Incremental processing with watermark expressions
- [Delete Inference](#) – Automatic soft-delete detection for missing records

Outputs

- [IO Output Modes](#) – File paths vs Unity Catalog table outputs

Other

- [Log Level Configuration](#) – Configuring log levels for processing output
- [Microsoft Fabric Compatibility](#) – Compatibility report for Microsoft Fabric runtime

For additional details or questions, please contact the maintainer.

Release Notes

Breaking Changes

- **Renamed** `DirectoryJsonMetadataSettings` **to** `JsonFolderMetadataSettings`.

This is not backward compatible. Update all references from `DirectoryJsonMetadataSettings` to `JsonFolderMetadataSettings`.

Entity Configuration

This document describes how to configure entities in Datalake Foundation, including column definitions, field roles, and settings hierarchy.

Entity Structure

An entity represents a data table or dataset that flows through the bronze-to-silver pipeline. Each entity is defined in the metadata JSON with the following properties:

```
{
  "id": 1,
  "name": "customer",
  "group": "sales",
  "destination": "dim_customer",
  "enabled": true,
  "secure": false,
  "connection": "AdventureWorksSql",
  "connectiongroup": "erp_group",
  "processtype": "merge",
  "watermark": [],
  "columns": [],
  "settings": {},
  "transformations": []
}
```

Entity Properties

Property	Type	Required	Description
<code>id</code>	Integer	Yes	Unique identifier for the entity
<code>name</code>	String	Yes	Source entity name (used in bronze path)
<code>group</code>	String	No	Logical grouping for batch processing
<code>destination</code>	String	No	Target name in silver layer (defaults to <code>name</code>)
<code>enabled</code>	Boolean	No	Whether processing is enabled (default: <code>true</code>)
<code>secure</code>	Boolean	No	Use secure container suffix paths (default: <code>false</code>)
<code>connection</code>	String	Yes	Reference to a connection definition
<code>connectiongroup</code>	String	No	Connection-level grouping

Property	Type	Required	Description
<code>processtype</code>	String	Yes	Processing strategy: <code>full</code> , <code>merge</code> , <code>historic</code> , or <code>stream</code>
<code>watermark</code>	Array	No	Watermark column definitions for incremental processing
<code>columns</code>	Array	No	Column definitions with types and roles
<code>settings</code>	Object	No	Entity-specific settings (override connection/environment)
<code>transformations</code>	Array	No	Custom transformation expressions

Column Definitions

Columns define the schema and behavior of entity fields. Each column can have multiple field roles that determine how it's used during processing.

```
{
  "columns": [
    {
      "name": "customer_id",
      "newname": "CustomerID",
      "datatype": "integer",
      "fieldroles": ["businesskey"],
      "expression": ""
    },
    {
      "name": "",
      "newname": "Region",
      "datatype": "string",
      "fieldroles": ["calculated", "partition"],
      "expression": "'EMEA'"
    }
  ]
}
```

Column Properties

Property	Type	Description
<code>name</code>	String	Source column name (empty for calculated columns)
<code>newname</code>	String	Target column name (rename during processing)
<code>datatype</code>	String	Target data type for casting
<code>fieldroles</code>	Array	List of roles this column fulfills

Property	Type	Description
<code>expression</code>	String	Spark SQL expression for calculated columns

Supported Data Types

- `string` - StringType
- `integer` - IntegerType
- `long` - LongType
- `float` - FloatType
- `double` - DoubleType
- `boolean` - BooleanType
- `date` - DateType
- `timestamp` - TimestampType
- `decimal(precision, scale)` - DecimalType (e.g., `decimal(18,2)`)

Field Roles

Field roles define how columns are used during processing. A column can have multiple roles.

`businesskey`

Columns marked as `businesskey` form the natural key of the entity. They are used to:

- Generate the primary key hash (`PK_<destination>`)
- Match records during merge operations
- Detect duplicates (throws `DuplicateBusinesskeyException`)

```
{
  "name": "customer_id",
  "fieldroles": ["businesskey"]
}
```

Multiple columns can be business keys - they are combined to form a composite key:

```
{
  "columns": [
    { "name": "region", "fieldroles": ["businesskey"] },
    { "name": "customer_id", "fieldroles": ["businesskey"] }
  ]
}
```

`partition`

Columns marked as `partition` are used as Delta Lake partition columns. This affects:

- How data is physically organized on storage
- Partition pruning during merge operations
- Dynamic partition overwrite behavior

```
{
  "name": "year",
  "fieldroles": ["partition"]
}
```

calculated

Calculated columns are derived from expressions rather than source data. When a column has **calculated** role:

- The **name** field can be empty
- The **expression** field contains a Spark SQL expression
- The column is added to the DataFrame during transformation

```
{
  "name": "",
  "newname": "ProcessedDate",
  "datatype": "date",
  "fieldroles": ["calculated"],
  "expression": "current_date()"
}
```

Combining Roles

A column can have multiple roles. For example, a calculated column that is also part of the business key:

```
{
  "name": "",
  "newname": "Administration",
  "datatype": "integer",
  "fieldroles": ["calculated", "businesskey", "partition"],
  "expression": "950"
}
```

Settings Hierarchy

Settings can be defined at three levels, with more specific levels overriding general ones:

```
Environment Settings (most general)
  ■■■ Connection Settings
    ■■■ Entity Settings (most specific)
```

Environment Settings

Defined in the **environment** block:

```
{
  "environment": {
    "name": "production",
    "root_folder": "/mnt/datalake",
    "timezone": "Europe/Amsterdam",
    "raw_path": "${connection}/${entity}",
    "bronze_path": "${connection}/${entity}",
    "silver_path": "${connection}/${destination}",
    "systemfield_prefix": "dlf_",
    "output_method": "paths"
  }
}
```

Connection Settings

Defined in each connection's **settings** block:

```
{
  "connections": [
    {
      "name": "AdventureWorksSql",
      "enabled": true,
      "settings": {
        "server": "sql-server.example.com",
        "database": "AdventureWorks2022"
      }
    }
  ]
}
```

Entity Settings

Defined in each entity's **settings** block (overrides connection settings):

```
{
  "entities": [
    {
      "id": 1,
      "name": "customer",
      "settings": {
        "silver_table": "silver_sales.dim_customer",
        "delete_missing": true
      }
    }
  ]
}
```

Common Settings

Setting	Level	Description
raw_path	Environment/Entity	Path pattern for raw files
bronze_path	Environment/Entity	Path pattern for bronze layer
silver_path	Environment/Entity	Path pattern for silver layer
bronze_table	Entity	Unity Catalog table name for bronze
silver_table	Entity	Unity Catalog table name for silver
output_method	Environment/Entity	Output type: paths or catalog
delete_missing	Entity	Enable delete inference (see Delete Inference)

Streaming Settings

When **processtype** is set to **stream**, the following additional settings configure the streaming behavior:

Setting	Level	Required	Default	Description
<code>stream_source</code>	Entity	No	<code>rate</code>	Streaming source format: <code>parquet</code> , <code>json</code> , <code>csv</code> , <code>orc</code> , <code>avro</code> , <code>kafka</code> , <code>kinesis</code> , or <code>rate</code>
<code>stream_write_strategy</code>	Entity	No	<code>merge</code>	Write strategy for micro-batches: <code>full</code> , <code>merge</code> , <code>delta</code> , or <code>historic</code>
<code>stream_trigger</code>	Entity	No	<code>processingTime</code>	Trigger type: <code>processingTime</code> , <code>availableNow</code> , or <code>once</code> (deprecated)
<code>stream_trigger_interval</code>	Entity	No	<code>0 seconds</code>	Trigger interval (only applies to <code>processingTime</code> trigger)
<code>stream_path</code>	Entity	Conditional	—	Source path (required for file-based sources: <code>parquet</code> , <code>json</code> , <code>csv</code> , <code>orc</code> , <code>avro</code>)
<code>checkpoint_location</code>	Entity	Conditional	Auto-computed	Checkpoint directory (required for <code>catalog</code> output, auto-computed for <code>paths</code> output)
<code>stream_max_offsets</code>	Entity	No	—	Maximum offsets per trigger (maps to <code>maxOffsetsPerTrigger</code>)
<code>stream_option.*</code>	Entity	No	—	Passthrough options to the stream reader (prefix stripped)
<code>{source}.*</code>	Entity	No	—	Source-specific options (e.g., <code>kafka.bootstrap.servers</code>)

See [Streaming Processing](#) for detailed documentation.

Expression Variables

Path and table settings support [expression variables](#):

Variable	Description
<code>\${connection}</code>	Connection name
<code>\${entity}</code>	Entity source name
<code>\${destination}</code>	Entity destination name
<code>\${today}</code>	Current date (yyyyMMdd format)
<code>\${settings_<key>}</code>	Value of any setting by key

Example:

```
{
  "silver_table": "silver_${connection}.${destination}",
  "bronze_path": "${connection}/${entity}/${today}"
}
```

Entity Transformations

Transformations allow injecting custom [Spark SQL expressions](#) during processing. Each transformation is an array of SELECT expressions applied to the DataFrame:

```
{
  "transformations": [
    ["customer_id", "concat(first_name, ' ', last_name) as FullName", "upper(region) as Region"],
    ["*", "year(order_date) as OrderYear"]
  ]
}
```

Each array represents one transformation step. Only columns listed in the expression array are kept in the output — use `"*"` to include all existing columns alongside new expressions. Multiple steps are applied in sequence.

Transformations are applied early in the processing pipeline, before system columns are added.

Example: Complete Entity Configuration

```
{
  "id": 42,
  "name": "sales_order",
  "group": "sales",
  "destination": "fact_sales",
  "enabled": true,
  "connection": "AdventureWorksSql",
  "processtype": "merge",
  "watermark": [
    {
      "column_name": "ModifiedDate",
      "operation": "and",
      "expression": "'${last_value}'"
    }
  ],
  "columns": [
    {
      "name": "SalesOrderID",
      "fieldroles": ["businesskey"]
    },
    {
      "name": "OrderDate",
      "datatype": "date",
      "fieldroles": ["partition"]
    },
    {
      "name": "TotalDue",
      "datatype": "decimal(18,2)",
      "fieldroles": []
    },
    {
      "name": "",
      "newname": "LoadDate",
      "datatype": "timestamp",
      "fieldroles": ["calculated"],
      "expression": "current_timestamp()"
    }
  ],
  "settings": {
    "silver_table": "silver_sales.fact_sales",
    "delete_missing": true
  },
  "transformations": []
}
```

See Also

- [Expression Languages](#)
- [Processing Strategies](#)
- [Watermarks](#)
- [IO Output Modes](#)

Metadata Sources

Datalake Foundation supports multiple sources for loading entity configuration metadata. This document describes each source type and how to configure them.

Overview

All metadata sources extend `DatalakeMetadataSettings` and provide the same configuration structure. The difference is where the JSON configuration is loaded from:

Source	Class	Use Case
JSON File	<code>JsonMetadataSettings</code>	Single configuration file
JSON Folder	<code>JsonFolderMetadataSettings</code>	Split configuration across files
SQL Server	<code>SqlMetadataSettings</code>	Central database-driven configuration
String	<code>StringMetadataSettings</code>	Programmatic/embedded configuration

JSON File Source

The most common approach - load configuration from a single JSON file.

Usage

```
import datalake.metadata._

val settings = new JsonMetadataSettings()
settings.initialize("/path/to/metadata.json")

val metadata = new Metadata(settings)
```

File Structure

The JSON file must contain three top-level sections:

```
{
  "environment": {
    "name": "production",
    "root_folder": "/mnt/datalake",
    "timezone": "Europe/Amsterdam",
    "raw_path": "/${connection}/${entity}",
    "bronze_path": "/${connection}/${entity}",
    "silver_path": "/${connection}/${destination}"
  },
  "connections": [
    {
      "name": "SalesDB",
      "enabled": true,
      "settings": {}
    }
  ],
  "entities": [
    {
      "id": 1,
      "name": "customer",
      "connection": "SalesDB",
      "processtype": "merge",
      "columns": [],
      "watermark": [],
      "settings": {},
      "transformations": []
    }
  ]
}
```

JSON Folder Source

Split configuration across multiple JSON files in a directory. Files are merged together, allowing modular organization.

Usage

```
import datalake.metadata._

val settings = new JsonFolderMetadataSettings()
settings.initialize("/path/to/config/directory")

val metadata = new Metadata(settings)
```

Directory Structure

```
config/
■■■ environment.json
■■■ connections.json
■■■ entities_sales.json
■■■ entities_finance.json
■■■ entities_hr.json
```

Each file contributes to the merged configuration:

environment.json

```
{
  "environment": {
    "name": "production",
    "root_folder": "/mnt/datalake",
    "timezone": "UTC",
    "raw_path": "/${connection}/${entity}",
    "bronze_path": "/${connection}/${entity}",
    "silver_path": "/${connection}/${destination}"
  }
}
```

connections.json

```
{
  "connections": [
    {"name": "SalesDB", "enabled": true, "settings": {}},
    {"name": "FinanceDB", "enabled": true, "settings": {}}
  ]
}
```

entities_sales.json

```
{
  "entities": [
    {"id": 1, "name": "customer", "connection": "SalesDB", "processtype": "merge", "columns": [], "watermark": null},
    {"id": 2, "name": "orders", "connection": "SalesDB", "processtype": "merge", "columns": [], "watermark": null}
  ]
}
```

Files are merged in filesystem order. Arrays with the same key are combined.

SQL Server Source

Load configuration from a SQL Server function. Useful for centralized configuration management.

Usage

```
import datalake.metadata._

val sqlSettings = SqlServerSettings(
  server = "sql-server.database.windows.net",
  port = 1433,
  database = "ConfigDB",
  username = "config_reader",
  password = "your_password"
)

val settings = SqlMetadataSettings(sqlSettings)
val metadata = new Metadata(settings)
```

SQL Server Setup

The library expects a function `cfg.fnGetFoundationConfig()` that returns a single row with a `config` column containing the full JSON configuration:

```

CREATE FUNCTION cfg.fnGetFoundationConfig()
RETURNS TABLE
AS
RETURN
(
    SELECT CAST(
        (SELECT
            (SELECT name, root_folder, timezone, raw_path, bronze_path, silver_path
              FROM cfg.Environment FOR JSON PATH, WITHOUT_ARRAY_WRAPPER) AS environment,
            (SELECT name, enabled, settings
              FROM cfg.Connections FOR JSON PATH) AS connections,
            (SELECT id, name, [group], destination, enabled, connection, processtype,
              (SELECT column_name, operation, operation_group, expression
                FROM cfg.Watermarks w WHERE w.entity_id = e.id FOR JSON PATH) AS watermark,
              (SELECT name, newname, datatype, fieldroles, expression
                FROM cfg.Columns c WHERE c.entity_id = e.id FOR JSON PATH) AS [columns],
              (SELECT * FROM cfg.EntitySettings s WHERE s.entity_id = e.id FOR JSON PATH, WITHOUT_ARRAY_WRAPPER) AS entity_settings,
              (SELECT column, expression
                FROM cfg.Transformations t WHERE t.entity_id = e.id FOR JSON PATH) AS transformations
            FROM cfg.Entities e FOR JSON PATH) AS entities
        FOR JSON PATH, WITHOUT_ARRAY_WRAPPER)
    AS NVARCHAR(MAX)) AS config
)

```

Connection String

The library builds a connection string with these defaults:

- Encryption enabled
- Trust server certificate disabled
- Azure SQL compatible hostname certificate
- 30 second login timeout

Security Recommendation

Avoid hardcoding credentials in your application. Use a Databricks secret scope backed by Azure Key Vault to retrieve **username** and **password** at runtime:

```

val sqlSettings = SqlServerSettings(
    server = "sql-server.database.windows.net",
    port = 1433,
    database = "ConfigDB",
    username = dbutils.secrets.get(scope = "keyvault-scope", key = "sql-username"),
    password = dbutils.secrets.get(scope = "keyvault-scope", key = "sql-password")
)

```

String Source

For programmatic configuration or testing. Pass the JSON configuration directly as a string.

Usage

```
import datalake.metadata._

val configJson = """
{
  "environment": {
    "name": "test",
    "root_folder": "/data",
    "timezone": "UTC",
    "raw_path": "raw",
    "bronze_path": "bronze",
    "silver_path": "silver"
  },
  "connections": [],
  "entities": []
}
"""

val settings = new StringMetadataSettings()
settings.initialize(configJson)

val metadata = new Metadata(settings)
```

Factory Method

A convenience factory method is available:

```
val settings = StringMetadataSettings(configJson)
```

Common Operations

Once metadata is loaded, the same operations are available regardless of source:

Get Entity by ID

```
val entity = metadata.getEntity(42)
// Returns Entity or throws EntityNotFoundException
```

Get Multiple Entities by ID

```
val entities = settings.getEntitiesById(Array(1, 2, 3))
// Returns Option[List[Entity]]
```

Get Entities by Connection

```
val connection = metadata.getConnection("SalesDB")
val entities = settings.getConnectionEntities(connection)
// Returns List[Entity]
```

Get Entities by Group

```
val group = metadata.getEntityGroup("sales")
val entities = settings.getGroupEntities(group)
// Returns List[Entity]
```

Get Entities by Connection Group

```
val connGroup = metadata.getEntityConnectionGroup("erp_systems")
val entities = settings.getConnectionGroupEntities(connGroup)
// Returns List[Entity]
```

Get Environment Settings

```
val env = metadata.getEnvironment
println(env.RootFolder)
println(env.Timezone)
```

Error Handling

The metadata system throws specific exceptions:

Exception	Cause
<code>MetadataNotInitializedException</code>	Accessing metadata before <code>initialize()</code>
<code>EntityNotFoundException</code>	Entity ID not found
<code>ConnectionNotFoundException</code>	Connection name not found
<code>DatalakeException</code>	Duplicate entity IDs in configuration

See Also

- [Entity Configuration](#)
- [Processing Strategies](#)

Expression Languages

Datalake Foundation uses two different expression languages depending on context. This document describes both and where each is used.

Overview

Language	Used For	Evaluated By
Spark SQL expressions	Calculated columns, transformations	Apache Spark
Scala expressions	Path/table settings, watermark values	Scala runtime

Spark SQL Expressions

Calculated columns and entity transformations use **Apache Spark SQL expression syntax**. These expressions support all Spark SQL built-in functions, column references, literals, arithmetic, string operations, conditional logic, and more.

Calculated Columns

When a column has the **calculated** field role, its **expression** field contains a Spark SQL expression that produces the column value.

```
{
  "name": "",
  "newname": "Region",
  "datatype": "string",
  "fieldroles": ["calculated"],
  "expression": "'EMEA'"
}
```

The expression can reference other columns in the DataFrame:

```
{
  "name": "",
  "newname": "FullName",
  "datatype": "string",
  "fieldroles": ["calculated"],
  "expression": "concat(first_name, ' ', last_name)"
}
```

More examples:

Expression	Result
"current_date()"	Today's date
"current_timestamp()"	Current timestamp
"'EMEA'"	String literal EMEA
"upper(name)"	Column name in uppercase

Expression	Result
<code>"year(order_date)"</code>	Year extracted from <code>order_date</code> column
<code>"coalesce(phone, 'N/A')"</code>	<code>phone</code> column with fallback
<code>"950"</code>	Integer literal <code>950</code>

Entity Transformations

Transformations apply Spark SQL `SELECT` expressions to reshape the entire `DataFrame`. Each transformation is defined as an array of column expressions, similar to a SQL `SELECT` clause.

```
{
  "transformations": [
    ["customer_id", "upper(name) as Name", "concat(city, ', ', country) as Location"],
    ["*", "year(order_date) as OrderYear"]
  ]
}
```

Each array in the list represents one transformation step. Multiple steps are applied in sequence. A transformation step replaces the current columns — only columns listed in the expression array are kept (use `"*"` to include all existing columns alongside new ones).

A single-expression transformation can also be specified as a plain string:

```
{
  "transformations": [
    ["*", "upper(name) as Name"]
  ]
}
```

Spark SQL Reference

For a complete list of available functions, see the [Spark SQL Built-in Functions](#) reference.

Scala Expressions

Path settings, table settings, and watermark values use **Scala string interpolation**. Variables are referenced with `${variable}` syntax and are resolved at runtime. Any valid Scala expression that produces a string is supported.

Path and Table Settings

Path and table settings support the following variables:

Variable	Description
<code>\${connection}</code>	Connection name
<code>\${entity}</code>	Entity source name
<code>\${destination}</code>	Entity destination name
<code>\${today}</code>	Current date in <code>yyyyMMdd</code> format
<code>\${settings_<key>}</code>	Value of any setting by key

Examples:

```
{
  "silver_table": "silver_${connection}.${destination}",
  "bronze_path": "${connection}/${entity}/${today}",
  "silver_path": "${settings_database}/${destination}"
}
```

These expressions are evaluated whenever path or table names are resolved, so `${today}` always reflects the current processing date.

Watermark Expressions

Watermark expressions use a different set of variables and also support Java time libraries for date arithmetic.

Variable	Description
<code>\${last_value}</code>	The last processed watermark value
<code>\${watermark}</code>	Alias for <code>\${last_value}</code>
<code>\${b19_epoch_day}</code>	Days since 1900-01-01 (for date-based systems)

Available Java time classes:

- `java.time.LocalDate`
- `java.time.LocalDateTime`
- `java.time.LocalTime`
- `java.time.format.DateTimeFormatter`

A pre-defined `defaultFormat` object is available: `DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss.S")`.

Examples:

```
// Simple last value passthrough
{ "expression": "${last_value}" }

// Date arithmetic: 7 days before last value
{ "expression": "${LocalDate.parse(last_value).minusDays(7)}" }

// Epoch day calculation
{ "expression": "${b19_epoch_day - 1}" }

// Reformat the last value
{ "expression": "${LocalDateTime.parse(last_value, defaultFormat).format(DateTimeFormatter.ISO_LOCAL_DATE)}" }
```

For full watermark configuration details, see [Watermarks](#).

See Also

- [Entity Configuration](#)
- [Watermarks](#)
- [Spark SQL Built-in Functions](#)

Log Level Configuration

The Data Lake Foundation library now supports configurable log levels through the metadata configuration file.

Configuration

Add the `log_level` field to the `environment` section of your metadata JSON:

```
{
  "environment": {
    "name": "PRODUCTION",
    "timezone": "UTC",
    "root_folder": "/data",
    "raw_path": "raw",
    "bronze_path": "bronze",
    "silver_path": "silver",
    "secure_container_suffix": "",
    "log_level": "INFO"
  },
  ...
}
```

Supported Log Levels

The following log levels are supported (in order of verbosity):

- **TRACE** - Most verbose, shows all log messages
- **DEBUG** - Detailed diagnostic information
- **INFO** - General information about application execution
- **WARN** - Warning messages for potentially harmful situations
- **ERROR** - Error messages for error conditions
- **FATAL** - Very severe error events that may cause termination
- **OFF** - No logging output

Default Behavior

If `log_level` is not specified in the environment configuration, the library defaults to **WARN** level.

Examples

Production Environment

```
{
  "environment": {
    "log_level": "INFO"
  }
}
```

Development Environment

```
{
  "environment": {
    "log_level": "DEBUG"
  }
}
```

Quiet Environment

```
{
  "environment": {
    "log_level": "WARN"
  }
}
```

This configuration applies to all loggers created through the Data Lake Foundation's **DataLakeLogManager**.

Processing Strategies

Datalake Foundation provides four processing strategies for moving data from bronze to silver layer. This document explains each strategy, when to use it, and how it behaves.

Overview

Strategy	Use Case	Behavior
Full	Initial loads, complete refreshes	Overwrites silver table with partition pruning
Merge	Incremental delta processing	Upserts based on primary key and source hash
Historic	SCD Type 2 tracking	Maintains version history with temporal columns
Stream	Continuous micro-batch ingestion	Structured Streaming with pluggable write strategies

Common Processing Pipeline

All strategies share a common transformation pipeline before writing:

1. **Inject transformations** - Apply custom expressions from entity config
2. **Add calculated columns** - Create derived columns with expressions
3. **Compute source hash** - SHA-256 of all columns for change detection
4. **Add temporal columns** - ValidFrom, ValidTo, IsCurrent (Historic only)
5. **Add filename column** - Track source slice file
6. **Generate primary key** - Hash of business key columns
7. **Cast columns** - Convert to target data types
8. **Rename columns** - Apply column name mappings
9. **Add deleted flag** - Soft-delete tracking column
10. **Add lastSeen timestamp** - Track when record was last processed
11. **Normalize column names** - Standardize naming format
12. **Cache DataFrame** - Cache the result for reuse during processing

Note: For streaming processing, step 12 (Cache DataFrame) is automatically skipped because micro-batch DataFrames are short-lived.

Full Strategy

The simplest strategy - performs a complete overwrite of the silver table.

Configuration

```
{
  "id": 1,
  "name": "reference_data",
  "processtype": "full",
  ...
}
```

Behavior

1. Reads the source parquet slice from bronze layer
2. Applies the transformation pipeline
3. Writes to silver using `SaveMode.Overwrite`
4. Uses dynamic partition overwrite mode (only replaces partitions present in source)

When to Use

- Initial data loads
- Reference/dimension tables that are small and fully refreshed
- Recovery scenarios requiring complete reload
- Tables without incremental change tracking

Usage

```
val processing = new Processing(entity, "2025-07-01-slice.parquet")
processing.Process(Full)
// or
processing.Process() // Uses entity's configured processtype
```

Partition Handling

With dynamic partition overwrite mode, only partitions present in the source slice are replaced:

```
Source slice contains: year=2024, year=2025
Silver table has: year=2023, year=2024, year=2025

Result: year=2024 and year=2025 are replaced
        year=2023 is preserved
```

Merge Strategy

Incremental processing using Delta Lake MERGE operations. Handles inserts, updates, and soft deletes.

Configuration

```
{
  "id": 2,
  "name": "transactions",
  "processtype": "merge",
  "watermark": [
    {
      "column_name": "ModifiedDate",
      "operation": "and",
      "expression": "'${last_value}'"
    }
  ],
  ...
}
```

Behavior

- 1. First run:** Falls back to Full strategy (no existing table to merge into)
- 2. Subsequent runs:**
 - Matches records by primary key
 - Applies partition filters to limit merge scope
 - Executes merge with these rules:

Condition	Action
Source deleted = true	Update target's deleted flag and lastSeen
Source hash $\neq$ Target hash	Update all columns
Source hash = Target hash	Update only lastSeen
No match in target	Insert new record

When to Use

- Large tables with incremental changes
- Systems that provide change data capture (CDC)
- Tables where you need upsert semantics
- When you want soft-delete support

Metrics

Metrics are reported for incremental runs. On first run, processing diverts to the Full strategy.

After processing, the strategy logs detailed metrics:

Metric	Description
recordsInSlice	Total records in the source slice
inserted	New records added (no matching primary key in target)
updated	All matched records, including both data changes and lastSeen-only touches
deleted	Source records that arrived with the deleted flag set to true. This is a source-side metric and does not include records removed via delete inference

The metric identity holds: `inserted + updated + deleted = recordsInSlice`

Note: Breaking change from previous versions. Earlier versions reported separate `updated` (source hash changed) and `touched` (source hash unchanged, only lastSeen updated) metrics. These have been combined into a single `updated` metric. The `touched` metric is always reported as 0 and should not be relied upon. If you have monitoring dashboards or alerting that depends on the old `updated` vs `touched` distinction, you must update them to use the combined `updated` metric. Users who need to distinguish real data changes from touch-only updates should compare source hashes independently.

Delete Inference

When `delete_missing` is enabled, records in the target that are missing from the source (within the watermark window) are automatically soft-deleted. See [Delete Inference](#) for details.

Historic Strategy (SCD Type 2)

Implements Slowly Changing Dimensions Type 2, maintaining a complete version history.

Configuration

```
{
  "id": 3,
  "name": "customer",
  "processtype": "historic",
  ...
}
```

System Columns

The Historic strategy adds temporal tracking columns:

Column	Type	Description
<code>{prefix}ValidFrom</code>	Timestamp	When this version became active
<code>{prefix}ValidTo</code>	Timestamp	When this version was superseded (null = current)
<code>{prefix}IsCurrent</code>	Boolean	Whether this is the current version
<code>{prefix}SourceHash</code>	String	Hash for change detection

Behavior

- 1. First run:** Falls back to Full strategy
- 2. Subsequent runs:**
 - Matches on primary key WHERE `IsCurrent = true`
 - When source hash differs from target:
 - Close current version: Set `ValidTo` and `IsCurrent = false`
 - Insert new version with updated values
 - New records (no PK match) are inserted directly

Version Lifecycle

```
Initial load:
  Customer 123: {data_v1, ValidFrom=2024-01-01, ValidTo=null, IsCurrent=true}

Update received:
  Customer 123: {data_v2, ...}

After processing:
  Customer 123: {data_v1, ValidFrom=2024-01-01, ValidTo=2024-06-15, IsCurrent=false}
  Customer 123: {data_v2, ValidFrom=2024-06-15, ValidTo=null, IsCurrent=true}
```

When to Use

- Dimension tables requiring audit trail
- Regulatory/compliance requirements for history
- Analytics needing point-in-time snapshots
- Master data where changes must be tracked

Metrics

Metrics are reported for incremental runs. On first run, processing diverts to the Full strategy.

After processing, the strategy logs detailed metrics:

Metric	Description
recordsInSlice	Total records in the source slice
inserted	New records with no matching primary key in the target
updated	Records where the source hash differs from the current target version. Each updated record closes the previous version (sets ValidTo and IsCurrent=false) and appends a new current version. The updated count directly equals the number of new historical versions created in this processing run.
unchanged	Records that matched a current target record with the same hash. No new version is created.
deleted	Records automatically soft-deleted because they were missing from the source (via delete inference). This is a target-side metric and is not included in the source-side identity.

Source-side identity: **inserted + updated + unchanged = recordsInSlice**

Note: The **deleted** metric has different semantics between Merge and Historic strategies. In Merge, **deleted** counts source records arriving with the deleted flag (source-side). In Historic, **deleted** counts target records removed via delete inference (target-side). These values are not directly comparable. Unlike Merge metrics, Historic metrics also include an **unchanged** count for records that matched but did not require a new version.

Querying Historic Data


```
-- Current state
SELECT * FROM silver.customer WHERE dlf_IsCurrent = true

-- Point-in-time snapshot
SELECT * FROM silver.customer
WHERE dlf_ValidFrom <= '2024-03-15'
  AND (dlf_ValidTo > '2024-03-15' OR dlf_ValidTo IS NULL)

-- Full history for a customer
SELECT * FROM silver.customer
WHERE customer_id = 123
ORDER BY dlf_ValidFrom
```

Stream Strategy

Continuous ingestion using Spark Structured Streaming with a pluggable write strategy applied per micro-batch.

Configuration

```
{
  "id": 5,
  "name": "event_stream",
  "processtype": "stream",
  "settings": {
    "stream_source": "kafka",
    "stream_write_strategy": "merge",
    "stream_trigger": "processingTime",
    "stream_trigger_interval": "30 seconds",
    "stream_path": null,
    "checkpoint_location": "/mnt/checkpoints/event_stream",
    "kafka.bootstrap.servers": "broker:9092",
    "kafka.subscribe": "events"
  }
}
```

Behavior

- Reads from a streaming source (file-based, Kafka, Kinesis, or rate)
- Applies the same transformation pipeline per micro-batch (caching automatically skipped)
- First micro-batch always writes using full/append mode
- Subsequent batches use the configured write strategy (**full**, **merge**, or **historic**)
- Returns a streaming query handle for lifecycle management

When to Use

- Continuous ingestion from message queues (Kafka, Kinesis)
- Near-real-time processing of file drops
- Low-latency data pipelines

Metrics

- **ProcessingSummary** emitted per micro-batch with **recordsInSlice**, **inserted**, **durationMs**
- Empty batches are skipped with a debug log

Processing Time Override

All strategies accept an optional processing time parameter. This controls the timestamp used for temporal columns.

Usage

```
val options = Map("processing.time" -> "2025-05-05T12:00:00")
val processing = new Processing(entity, "slice.parquet", options)
processing.Process()
```

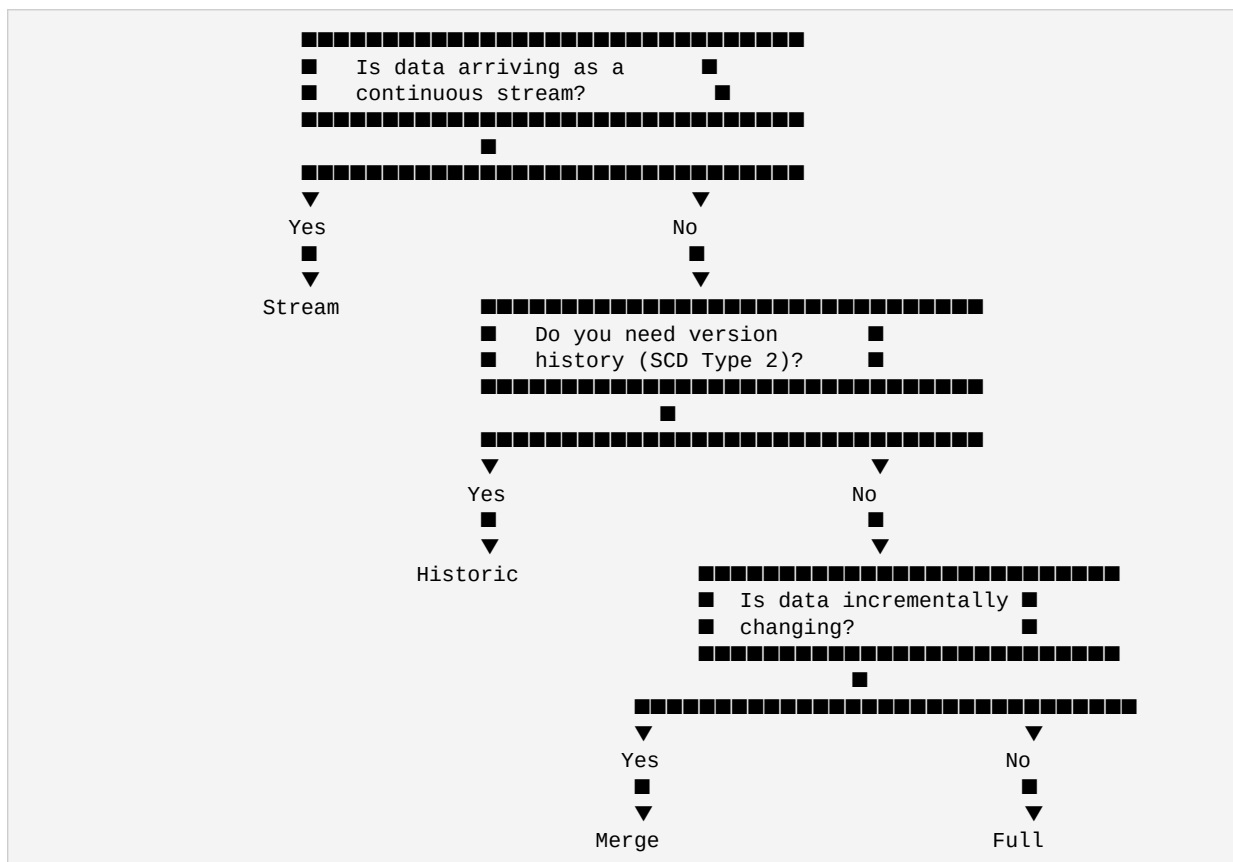
Behavior

- If not specified, uses current system time
- Must be ISO-8601 format
- Invalid timestamps log a warning and fall back to current time

Caution

The library does not validate temporal succession. Providing an earlier `processing.time` than previously processed data can result in inconsistent `ValidFrom/ValidTo` ranges in Historic mode.

Choosing a Strategy



Error Handling

Duplicate Business Keys

If the source slice contains duplicate business keys, processing fails with `DuplicateBusinesskeyException`. This prevents data quality issues in the silver layer.

Schema Drift

All strategies detect schema differences between source and target:

- Column additions are logged as warnings
- Column removals are logged as warnings
- Type mismatches are logged as warnings

Processing continues with the current source schema.

See Also

- [Entity Configuration](#)
- [Watermarks](#)
- [Delete Inference](#)
- [Streaming Processing](#)

Streaming Processing

Datalake Foundation's **Stream** strategy enables continuous ingestion from live data sources into Delta Lake silver tables. It is the first strategy that runs as a long-lived Spark Structured Streaming query rather than a one-shot batch operation.

Overview

Property	Value
Strategy name	<code>stream</code>
Process type	Continuous (Structured Streaming)
Return value	Returns a streaming query handle — the caller manages its lifecycle
Write modes	<code>full</code> , <code>merge</code> (default), <code>historic</code>
Checkpoint	Required — auto-computed for path-based destinations

The Stream strategy returns a streaming query handle instead of completing immediately like batch strategies. The caller is responsible for managing the query lifecycle (e.g., calling `query.awaitTermination()` or `query.stop()`).

The same transformation pipeline used by batch strategies runs inside each micro-batch. Schema inference, primary key generation, source hash computation, column casting, and renaming all apply identically. DataFrame caching is automatically skipped for streaming because micro-batch DataFrames are short-lived.

Configuration Reference

All streaming settings are defined in the entity's `settings` object alongside regular entity settings.

Setting	Type	Default	Description
<code>stream_source</code>	string	<code>"rate"</code>	Streaming source format. See Source Types .
<code>stream_write_strategy</code>	string	<code>"merge"</code>	How data is written per micro-batch: <code>full</code> , <code>merge</code> , <code>historic</code> . <code>"delta"</code> is accepted as an alias for <code>"merge"</code> .
<code>stream_trigger</code>	string	<code>"processingTime"</code>	Trigger type: <code>processingTime</code> , <code>availableNow</code> , or <code>once</code> (deprecated).

Setting	Type	Default	Description
<code>stream_trigger_interval</code>	string	"0 seconds"	Interval for <code>processingTime</code> trigger only (e.g. "30 seconds", "1 minute"). Ignored for other trigger types.
<code>stream_path</code>	string	—	Path to the source data. Required for file-based sources: <code>parquet</code> , <code>json</code> , <code>csv</code> , <code>orc</code> , <code>avro</code> .
<code>checkpoint_location</code>	string	Auto-computed	Checkpoint directory for the streaming query. Required for <code>TableLocation</code> destinations. Auto-computed for <code>PathLocation</code> as <code>{silver_path}/_checkpoints/{entity_name}</code> .
<code>stream_max_offsets</code>	string	—	Maximum number of offsets to process per trigger (maps to <code>maxOffsetsPerTrigger</code>). Useful for throttling Kafka or Kinesis sources.
<code>stream_option.<key></code>	string	—	Passthrough reader options. The <code>stream_option.</code> prefix is stripped before passing to the reader (e.g. <code>stream_option.header</code> becomes <code>header</code>).
<code><source>.<key></code>	string	—	Source-specific passthrough options keyed by source name (e.g. <code>kafka.bootstrap.servers</code> , <code>kafka.subscribe</code>).

Trigger Types

Trigger	Description	When to Use
<code>processingTime</code>	Runs micro-batches on a fixed interval. Use <code>stream_trigger_interval</code> to control frequency. "0 seconds" means run as fast as possible.	Continuous low-latency ingestion

Trigger	Description	When to Use
<code>availableNow</code>	Processes all available data then stops. Recommended replacement for <code>once</code> .	Scheduled batch-like runs over a streaming source
<code>once</code>	Processes one micro-batch then stops. Deprecated as of Spark 3.4+.	Legacy use only — prefer <code>availableNow</code>

Source Types

File-Based Sources

Supported formats: `parquet`, `json`, `csv`, `orc`, `avro`

- `stream_path` is **required** and must point to the directory containing source files.
- Schema is **automatically inferred** by reading the directory with `spark.read` before starting the stream. This requires at least one file to exist at stream start time; if schema inference fails a `DatalakeException` is thrown.
- New files dropped into the directory are picked up automatically by Spark's file source.

Message-Based Sources

Supported formats: `kafka`, `kinesis`

- `stream_path` is **not** used. Source connectivity is configured via source-specific passthrough options (e.g. `kafka.bootstrap.servers`, `kafka.subscribe`).
- Schema is determined by the message format returned by the source.

Test Source

The `rate` source (default) generates rows at a configurable rate with no external dependency. It is useful for testing streaming pipelines end-to-end without a real source.

Write Strategies

First Run Behavior

The first time a streaming query runs against a destination that has no existing Delta table, the first non-empty micro-batch is written using **full write mode** (Append to a new table). This is consistent with how batch strategies handle first runs.

For `historic` write strategy, the first-run batch additionally adds the SCD2 temporal columns (`ValidFrom`, `ValidTo`, `IsCurrent`) set to the current batch time.

After the first micro-batch completes, subsequent batches follow the normal per-strategy write logic.

Note: First-run detection is evaluated once before the stream starts and cleared after the first non-empty batch completes successfully.

Per-Strategy Write Logic (After First Run)

full

Appends the micro-batch to the destination using `SaveMode.Append`. Unlike the batch Full strategy (which overwrites), the streaming variant always appends to avoid overwriting data written by previous micro-batches.

merge / delta

Executes a Delta MERGE on each micro-batch, matching on primary key. The merge rules are:

Condition	Action
Source <code>deleted = true</code>	Update target's <code>deleted</code> flag and <code>lastSeen</code>
Source hash <code>!=</code> Target hash	Update all columns
Source hash <code>==</code> Target hash	Update only <code>lastSeen</code>
No match in target	Insert new record (only if <code>deleted = false</code>)

"delta" is accepted as a backward-compatible alias for "merge".

historic

Executes a two-phase SCD Type 2 MERGE per micro-batch:

1. Match on primary key WHERE `IsCurrent = true` and source hash differs — close current version by setting `ValidTo` to the current batch time and `IsCurrent = false`.
2. Insert all source records (new and changed) as new versions with `ValidFrom` set to the current batch time, `ValidTo = 2999-12-31`, and `IsCurrent = true`.

Important: Delete inference (`inferDeletesFromMissing`) is explicitly disabled in streaming historic mode. Because micro-batches do not represent complete data slices, it is impossible to determine which records are genuinely absent versus simply not included in this batch. A warning is logged if `inferDeletesFromMissing` is configured.

Checkpoint Management

Spark Structured Streaming requires a checkpoint location to track progress and enable fault tolerance.

Destination type	Behavior
<code>PathLocation</code>	If <code>checkpoint_location</code> is not set, a default is computed: <code>{silver_path}/_checkpoints/{entity_name}</code> . A warning is logged.
<code>TableLocation</code>	<code>checkpoint_location</code> must be set explicitly. If omitted, a <code>DataLakeException</code> is thrown at startup. Using <code>/tmp</code> as a fallback is unsafe for production and is explicitly rejected.

Micro-Batch Execution Flow

For each micro-batch triggered by the streaming query:

1. **Empty batch check** — if the batch is empty, it is skipped and a debug log entry is written. No write occurs.

2. Transform chain — the same transformation pipeline as batch strategies runs on each micro-batch (inject transformations, calculated columns, source hash, temporal columns, filename, primary key, cast, rename, deleted flag, lastSeen, normalize). DataFrame caching is automatically skipped.

3. Write — the micro-batch is written using the configured write strategy (**full**, **merge**, or **historic**) and the current first-run state.

4. Summary logging — processing metrics are logged for each completed micro-batch with record counts and duration.

5. Error handling — exceptions during micro-batch processing are logged and re-thrown. Spark will retry or stop the query depending on the configured restart policy.

Usage in Your Notebook

The streaming strategy returns a query handle that you manage in your notebook or job:

```
val entity = metadata.getEntity(42)
val processing = new Processing(entity, "") // streaming reads from its configured source, not a slice file
val query = processing.Process(Stream)

// Block until the stream is stopped or fails
query.foreach(_.awaitTermination())

// Or stop it explicitly
query.foreach(_.stop())
```

Batch strategies return no query handle — they complete immediately when processing finishes.

Limitations

- **Watermarks are ignored.** If watermark columns are configured on the entity, they are detected and a warning is logged, but watermark values are never read or written.
- **Streaming does not use bronze slice files or watermark tracking.** Each micro-batch receives its data directly from the configured streaming source.
- **Delete inference is disabled for historic.** `inferDeletesFromMissing` has no effect in streaming historic mode (see [Write Strategies](#) above).
- **DataFrame caching is managed automatically.** The streaming lifecycle handles caching differently from batch strategies — no manual intervention is needed.

Example Configuration

Kafka Entity (merge write strategy)


```
{
  "id": 10,
  "name": "orders_stream",
  "processtype": "stream",
  "settings": {
    "stream_source": "kafka",
    "stream_write_strategy": "merge",
    "stream_trigger": "processingTime",
    "stream_trigger_interval": "30 seconds",
    "stream_max_offsets": "10000",
    "checkpoint_location": "/mnt/checkpoints/orders_stream",
    "kafka.bootstrap.servers": "broker1:9092,broker2:9092",
    "kafka.subscribe": "orders-topic",
    "kafka.startingOffsets": "latest"
  },
  "businesskey": ["order_id"],
  "columns": [...]
}
```

File-Based Entity (historic write strategy, availableNow trigger)

```
{
  "id": 11,
  "name": "customers_historic_stream",
  "processtype": "stream",
  "settings": {
    "stream_source": "parquet",
    "stream_write_strategy": "historic",
    "stream_trigger": "availableNow",
    "stream_path": "/mnt/bronze/customers/",
    "stream_option.mergeSchema": "true"
  },
  "businesskey": ["customer_id"],
  "columns": [...]
}
```

See Also

- [Processing Strategies](#) — Full, Merge, and Historic batch strategies
- [Entity Configuration](#) — Full entity settings reference
- [Delete Inference](#) — How missing-record delete detection works in batch mode
- [Watermarks](#) — Watermark configuration (not applicable to streaming)

Watermarks

Watermarks enable incremental processing by tracking the last processed values. This document explains how to configure watermarks and write watermark expressions.

Overview

A watermark defines a column and expression that determines which records to process in each run. The library stores the last processed value and uses it to filter source data on subsequent runs.

Watermark Configuration

Watermarks are defined in the entity's `watermark` array:

```
{
  "id": 1,
  "name": "orders",
  "processtype": "merge",
  "watermark": [
    {
      "column_name": "ModifiedDate",
      "operation": "and",
      "operation_group": 0,
      "expression": "'${last_value}'"
    }
  ]
}
```

Properties

Property	Type	Description
<code>column_name</code>	String	The column to track for incremental processing
<code>operation</code>	String	Logical operator: <code>and</code> or <code>or</code>
<code>operation_group</code>	Integer	Group watermarks for complex conditions
<code>expression</code>	String	Dynamic expression for the watermark value

Expression System

Watermark expressions are evaluated at runtime using [Scala expressions](#). The expression produces the filter value used in processing.

Available Parameters

Parameter	Description
<code>\${last_value}</code>	The last processed watermark value

Parameter	Description
<code>\${watermark}</code>	Alias for <code>\${last_value}</code>
<code>\${b19_epoch_day}</code>	Days since 1900-01-01 (for date-based systems)
<code>\${reflex_now}</code>	Alias for <code>\${b19_epoch_day}</code>

Available Libraries

Expressions can use these Java time classes:

- `java.time.LocalDate`
- `java.time.LocalDateTime`
- `java.time.LocalTime`
- `java.time.format.DateTimeFormatter`

Pre-defined Objects

Object	Value
<code>defaultFormat</code>	<code>DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss.S")</code>
<code>b19_epoch_day</code>	Days from 1900-01-01 to today

Expression Examples

Simple Last Value

Use the last processed value directly:

```
{
  "column_name": "ModifiedDate",
  "operation": "and",
  "expression": "'${last_value}'"
}
```

Date Arithmetic

Process records from 7 days before the last value:

```
{
  "column_name": "EventDate",
  "operation": "and",
  "expression": "LocalDate.parse('${last_value}').minusDays(7).toString()"
}
```

Epoch Day Calculation

For systems using days since epoch:

```
{
  "column_name": "DayNumber",
  "operation": "and",
  "expression": "${b19_epoch_day} - 1"
}
```

Formatted Date

Parse and reformat the last value:

```
{
  "column_name": "ProcessDate",
  "operation": "and",
  "expression": "LocalDateTime.parse('${last_value}', defaultFormat).format(DateTimeFormatter.ISO_LOCAL_DATE)"
}
```

Multiple Watermarks

Entities can have multiple watermarks combined with logical operators.

AND Conditions

Both conditions must be met:

```
{
  "watermark": [
    {
      "column_name": "ModifiedDate",
      "operation": "and",
      "operation_group": 0,
      "expression": "'${last_value}'"
    },
    {
      "column_name": "Region",
      "operation": "and",
      "operation_group": 0,
      "expression": "'EMEA'"
    }
  ]
}
```

Generated filter: `ModifiedDate >= <value> AND Region >= 'EMEA'`

OR Conditions

Either condition triggers processing:

```
{
  "watermark": [
    {
      "column_name": "CreateDate",
      "operation": "or",
      "operation_group": 0,
      "expression": "'${last_value}'"
    },
    {
      "column_name": "UpdateDate",
      "operation": "or",
      "operation_group": 0,
      "expression": "'${last_value}'"
    }
  ]
}
```

Generated filter: `CreateDate >= <value> OR UpdateDate >= <value>`

Operation Groups

Group watermarks for complex logic:

```
{
  "watermark": [
    {
      "column_name": "ModifiedDate",
      "operation": "and",
      "operation_group": 0,
      "expression": "'${last_value}'"
    },
    {
      "column_name": "Status",
      "operation": "or",
      "operation_group": 1,
      "expression": "'PENDING'"
    },
    {
      "column_name": "Status",
      "operation": "or",
      "operation_group": 1,
      "expression": "'ACTIVE'"
    }
  ]
}
```

Generated filter: `ModifiedDate >= <value> AND (Status = 'PENDING' OR Status = 'ACTIVE')`

Watermark Lifecycle

First Run

On the first run (no previous watermark value stored):

1. Watermark filter is not applied
2. All source records are processed
3. Current max value is stored for next run

Subsequent Runs

1. Previous watermark value is retrieved

2. Expression is evaluated with `${last_value}`
3. Source is filtered using the watermark condition
4. After successful processing, new max value is stored

Watermark Storage

Watermark values are stored in a system table and associated with the entity ID. The `WatermarkData` class handles storage and retrieval.

Resetting Watermarks

To reprocess data, you can reset watermarks programmatically:

```
val entity = metadata.getEntity(42)

// Reset to null (will trigger full reprocess)
entity.Watermark.foreach(_.Reset)

// Reset to specific value
entity.Watermark.foreach(_.Reset("2024-01-01 00:00:00.0"))
```

Common Patterns

Timestamp-Based Incremental

```
{
  "column_name": "LastModified",
  "operation": "and",
  "expression": "'${last_value}'"
}
```

Sequence Number

```
{
  "column_name": "SeqNr",
  "operation": "and",
  "expression": "'${last_value}'"
}
```

Date with Buffer

Include a 3-day overlap to catch late-arriving data:

```
{
  "column_name": "EventDate",
  "operation": "and",
  "expression": "LocalDate.parse('${last_value}').minusDays(3).toString()"
}
```

Legacy Epoch Day Systems

For systems using days since 1900-01-01:

```
{
  "column_name": "BatchDay",
  "operation": "and",
  "expression": "${reflex_now}"
}
```

Watermarks and Delete Inference

Watermarks play a critical role in [Delete Inference](#). The watermark window defines the scope of records that should be present in the source slice. Records in the target within this window that are missing from the source are candidates for soft-delete.

Error Handling

If expression evaluation fails:

1. The error is logged
2. The watermark value returns **None**
3. Processing continues without that watermark filter

Common errors:

- Invalid date format in expression
- Missing **last_value** on first run (handled gracefully)
- Syntax errors in expressions

See Also

- [Expression Languages](#)
- [Processing Strategies](#)
- [Delete Inference](#)
- [Entity Configuration](#)

Delete Inference

Delete inference automatically soft-deletes records that exist in the target but are missing from the source slice. This document explains how the feature works, its configuration, and edge cases to be aware of.

Overview

When processing incremental data, source systems may not always send explicit delete records. The delete inference feature (**delete_missing**) detects these implicit deletes by comparing what records *should* exist in the current slice (based on the watermark window) against what actually exists.

Enabling Delete Inference

Enable delete inference in the entity settings:

```
{
  "id": 1,
  "name": "orders",
  "processtype": "merge",
  "settings": {
    "delete_missing": true
  },
  "watermark": [
    {
      "column_name": "ModifiedDate",
      "operation": "and",
      "expression": "'${last_value}'"
    }
  ]
}
```

Requirements:

- **delete_missing: true** in entity settings
- At least one watermark column defined
- Merge or Historic processing strategy

How It Works

Step 1: Define the Watermark Window

The watermark window defines the range of records expected in the current slice:

```
Previous watermark value: 2024-06-01 (last processed)
Current watermark value: 2024-06-15 (max in current slice)

Window: ModifiedDate >= '2024-06-01' AND ModifiedDate <= '2024-06-15'
```

Step 2: Identify Missing Records

During the merge operation, records in the target that:

1. Fall within the watermark window

2. Match the partition filter (if applicable)
3. Are NOT already deleted
4. Have no matching record in the source

...are candidates for deletion.

Step 3: Soft Delete

Missing records are marked as deleted:

- `{prefix}deleted = true`
- `{prefix}lastSeen = <processing_time>`
- For Historic: `{prefix}IsCurrent = false, {prefix}ValidTo = <processing_time>`

Safety Filters

Delete inference applies multiple safety filters to prevent accidental deletions:

Watermark Window

Only considers records within the expected data range. Records outside this window are not affected.

```
target.ModifiedDate >= '2024-06-01'  
AND target.ModifiedDate <= '2024-06-15'
```

Partition Filter

If the entity has partition columns, only records in partitions present in the source slice are considered.

```
target.Region IN ('EMEA', 'APAC') -- Partitions found in source
```

Not Already Deleted

Avoids repeatedly updating already-deleted records.

```
target.dlf_deleted = false
```

IsCurrent (Historic Only)

For SCD Type 2, only operates on current versions.

```
target.dlf_IsCurrent = true
```

Edge Cases

First Run (No Previous Watermark)

On the first run, there is no previous watermark value. Delete inference is safely disabled:

```
WARNING: inferDeletesFromMissing is enabled but no previous watermark values  
available. No deletes will be inferred. This is expected on first run.
```

NULL Watermark Values

Records with NULL values in watermark columns will NOT be considered for deletion. This is logged as a warning:

```
WARNING: Records with NULL values in watermark columns will NOT be
considered for deletion.
```

Missing Watermark Columns

If the watermark column doesn't exist in the target schema, that column is skipped:

```
WARNING: Watermark column 'ModifiedDate' not found in target schema;
skipping it for delete detection.
```

Gaps in Processing

Large gaps between the previous and current watermark may cause issues:

```
Previous: 2024-01-01
Current:  2024-12-31  (11 month gap)
```

If the source system only provides recent changes, records modified in the middle of this gap may be incorrectly marked as deleted. Design your watermark expressions to minimize gaps.

Strategy-Specific Behavior

Merge Strategy

When a record in the Silver table is not matched by any record in the current source batch (within the watermark window), the library marks it as soft-deleted by setting the `deleted` flag to `true` and updating `lastSeen` to the current processing timestamp.

Historic Strategy

When a record in the Silver table is not matched by any record in the current source batch (within the watermark window), the library closes the current version by:

- Setting `deleted` to `true`
- Setting `IsCurrent` to `false`
- Setting `ValidTo` to the current processing timestamp
- Updating `lastSeen` to the current processing timestamp

Logging

Delete inference logs key information:

```
INFO: Delete detection window for column 'ModifiedDate':  
      [2024-06-01 <= ModifiedDate <= 2024-06-15]  
  
INFO: inferDeletesFromMissing is active. Records in the watermark window  
      that are missing from the source will be marked as deleted.  
  
WARNING: Note: Records with NULL values in watermark columns will NOT be  
         considered for deletion.
```

Example Scenario

Source System Behavior

Orders system sends all orders modified since last sync. When an order is deleted, it simply stops appearing in subsequent syncs.

Configuration

```
{  
  "id": 10,  
  "name": "orders",  
  "processtype": "merge",  
  "watermark": [  
    {  
      "column_name": "LastModified",  
      "operation": "and",  
      "expression": "'${last_value}'"  
    }  
  ],  
  "columns": [  
    {"name": "OrderID", "fieldroles": ["businesskey"]},  
    {"name": "CustomerID", "fieldroles": []},  
    {"name": "LastModified", "fieldroles": []}  
  ],  
  "settings": {  
    "delete_missing": true  
  }  
}
```

Processing Flow

Run 1 (Initial)

- Source: Orders 1, 2, 3
- Previous watermark: None
- Delete inference: Disabled (first run)
- Result: All orders inserted, watermark set to max(LastModified)

Run 2 (Normal)

- Source: Orders 2, 3, 4
- Previous watermark: 2024-06-01
- Current watermark: 2024-06-15
- Delete inference: Active
- Order 1 is in window but missing from source → Soft deleted
- Order 4 is new → Inserted

Run 3 (With Late Data)

- Source: Orders 2, 3
- Previous watermark: 2024-06-15
- Current watermark: 2024-06-20
- Delete inference: Active
- Order 4 now in window but missing → Soft deleted

Best Practices

1. **Use appropriate watermark expressions** - Include buffer time if source has late-arriving data
2. **Monitor delete counts** - Unusual spikes may indicate issues
3. **Test with realistic data** - Verify behavior matches source system patterns
4. **Consider partitioning** - Reduces scope of delete detection
5. **Handle NULL watermarks** - Ensure source data has valid watermark values

Disabling Delete Inference

If delete inference causes issues, disable it by removing or setting `delete_missing: false`:

```
{
  "settings": {
    "delete_missing": false
  }
}
```

See Also

- [Watermarks](#)
- [Processing Strategies](#)
- [Entity Configuration](#)

IO Output Modes

Datalake Foundation supports two output modes: file paths and Unity Catalog tables. This document explains how to configure each mode and how they affect processing behavior.

Overview

Mode	Setting	Output Type	Use Case
Paths	<code>output_method:</code> <code>"paths"</code>	File system paths	Traditional data lakes, ADLS/S3
Catalog	<code>output_method:</code> <code>"catalog"</code>	Unity Catalog tables	Databricks Unity Catalog

Configuration Hierarchy

Output mode can be set at multiple levels:

Environment (default for all entities)
 ■■■ Connection (override for connection)
 ■■■ Entity (override for specific entity)

Paths Mode (Default)

Paths mode writes data to file system locations as Delta tables.

Environment Configuration

```
{
  "environment": {
    "name": "production",
    "root_folder": "/mnt/datalake",
    "raw_path": "${connection}/${entity}",
    "bronze_path": "${connection}/${entity}",
    "silver_path": "${connection}/${destination}",
    "output_method": "paths"
  }
}
```

Resulting Paths

For an entity with:

- Connection: `AdventureWorksSql`
- Name: `customer`
- Destination: `dim_customer`

Generated paths:

```
Raw:      /mnt/datalake/raw/AdventureWorksSql/customer
Bronze:   /mnt/datalake/bronze/AdventureWorksSql/customer
Silver:   /mnt/datalake/silver/AdventureWorksSql/dim_customer
```

Entity Override

Override paths at entity level:

```
{
  "id": 1,
  "name": "customer",
  "settings": {
    "bronze_path": "/custom/bronze/path/${entity}",
    "silver_path": "/custom/silver/path/${destination}"
  }
}
```

Catalog Mode

Catalog mode writes data to Unity Catalog managed tables.

Environment Configuration

```
{
  "environment": {
    "name": "production",
    "root_folder": "/mnt/datalake",
    "raw_path": "/${connection}/${entity}",
    "bronze_path": "/${connection}/${entity}",
    "silver_path": "/${connection}/${destination}",
    "output_method": "catalog"
  }
}
```

Default Table Names

When using catalog mode, default table names follow this pattern:

```
Bronze: bronze_<connection>.<entity>
Silver: silver_<connection>.<destination>
```

For an entity with:

- Connection: **AdventureWorksSql**
- Name: **customer**
- Destination: **dim_customer**

Generated tables:

```
Bronze: bronze_AdventureWorksSql.customer
Silver: silver_AdventureWorksSql.dim_customer
```

Entity Override

Specify custom table names:

```
{
  "id": 1,
  "name": "customer",
  "settings": {
    "bronze_table": "raw_data.bronze_customers",
    "silver_table": "curated_data.dim_customer"
  }
}
```

Mixed Mode

You can mix paths and catalog at the entity level. This is useful for transition scenarios or when different data zones use different storage patterns.

Example: Paths Bronze, Catalog Silver

```
{
  "environment": {
    "output_method": "paths"
  },
  "entities": [
    {
      "id": 1,
      "name": "customer",
      "settings": {
        "silver_table": "silver_sales.dim_customer"
      }
    }
  ]
}
```

Result:

- Bronze: `/mnt/datalake/bronze/AdventureWorksSql/customer` (path)
- Silver: `silver_sales.dim_customer` (catalog table)

Example: Environment Catalog with Path Override

```
{
  "environment": {
    "output_method": "catalog"
  },
  "entities": [
    {
      "id": 1,
      "name": "large_dataset",
      "settings": {
        "bronze_path": "/archive/bronze/${entity}",
        "silver_path": "/archive/silver/${destination}"
      }
    }
  ]
}
```

Result:

- Bronze: `/archive/bronze/large_dataset` (path)
- Silver: `/archive/silver/large_dataset` (path)

Per-Layer Configuration

Fine-grained control is available with `bronze_output` and `silver_output`:

```
{
  "environment": {
    "output_method": "paths",
    "bronze_output": "paths",
    "silver_output": "catalog"
  }
}
```

This configures:

- Bronze layer: File paths
- Silver layer: Unity Catalog tables

Expression Variables

Table and path settings support expression variables:

Variable	Description
<code>\${connection}</code>	Connection name
<code>\${entity}</code>	Entity source name
<code>\${destination}</code>	Entity destination name
<code>\${today}</code>	Current date (yyyyMMdd)
<code>\${settings_<key>}</code>	Any setting value

Examples:

```
{
  "silver_table": "silver_${connection}.${destination}",
  "bronze_path": "${connection}/${entity}/${today}"
}
```

Accessing Output in Code

Get Output Configuration

```
val entity = metadata.getEntity(42)
val output = entity.getOutput

output match {
  case Output(rawPath, bronze, silver) =>
    bronze match {
      case PathLocation(path) => println(s"Bronze path: $path")
      case TableLocation(table) => println(s"Bronze table: $table")
    }
    silver match {
      case PathLocation(path) => println(s"Silver path: $path")
      case TableLocation(table) => println(s"Silver table: $table")
    }
}
```


Get Paths (Legacy Compatibility)

```
val paths = entity.getPaths
println(paths.rawpath)
println(paths.bronzepath)
println(paths.silverpath)
```

Note: `getPaths` returns path strings even when catalog mode is configured. For catalog tables, it returns the underlying path if available.

Migration Considerations

When migrating from paths to catalog:

1. **Schema Registration:** Tables will be registered in the catalog
2. **Permissions:** Ensure proper Unity Catalog permissions
3. **Existing Data:** Data in existing paths won't be automatically migrated
4. **Dependencies:** Update downstream systems to reference catalog tables

Migration Strategy

1. Start with new entities using catalog mode
2. Gradually migrate existing entities
3. Use mixed mode during transition
4. Update documentation and access patterns

JSON Serialization

Entity JSON output reflects the configured output mode:

Paths mode:

```
{
  "paths": {
    "rawpath": "/data/raw/...",
    "bronzepath": "/data/bronze/...",
    "silverpath": "/data/silver/..."
  }
}
```

Catalog mode:

```
{
  "output": {
    "raw_path": "/data/raw/...",
    "bronze_table": "bronze_conn.entity",
    "silver_table": "silver_conn.entity"
  }
}
```

See Also

- [Entity Configuration](#)
- [Metadata Sources](#)

- Processing Strategies

Microsoft Fabric Compatibility

Investigation date: February 2026

Summary

Datalake Foundation can run on Microsoft Fabric. The recommended path is **Fabric Runtime 2.0**, which provides an exact version match for Spark 4.0.0, Scala 2.13.16, and Delta Lake 4.0.0. Runtime 2.0 is currently in Experimental Preview (since December 2025), with GA expected mid-to-late 2026.

Fabric Runtime Compatibility Matrix

Requirement	Runtime 1.3 (GA)	Runtime 2.0 (Experimental Preview)
Spark 4.0.0	NO (3.5.5)	YES (4.0.0)
Scala 2.13.16	NO (2.12.17)	YES (2.13.16)
Delta Lake 4.0.0	NO (3.2)	YES (4.0.0)
Java 17	NO (Java 11)	PARTIAL (Java 21, forward-compatible)
Custom JAR support	YES (Environment + inline)	LIMITED (inline only for now)
Delta MERGE	YES	NOT YET in early preview
SCD Type 2	YES	Depends on MERGE support
Dynamic partition overwrite	YES	YES (core Spark feature)

Route A: Runtime 2.0 (Recommended)

What works as-is

- All path-based operations (paths are metadata-driven, no hardcoded `dbfs:/`)
- `Full` processing strategy (dynamic partition overwrite)
- `SparkSession` creation (standard `getOrCreate()`)
- SQL metadata source (Azure SQL JDBC accessible from Fabric)
- JSON metadata sources
- Log4j logging (standard Log4j 2.x)
- commons-io (bundled)

Minor code changes needed

1. Unity Catalog config checks — Databricks-specific Unity Catalog config checks (`spark.databricks.unityCatalog.enabled`, `spark.databricks.defaultCatalog`) should be replaced with Fabric-equivalent or made configurable. Currently falls back gracefully to path-based writes, so non-blocking.

- 2. Java 21 testing** — Verify JAR compiled for Java 17 runs correctly on Java 21 (forward-compatible bytecode, but should be tested).
- 3. Scala compiler on classpath** — Verify `scala-compiler` (used by `Expressions.scala` `ToolBox`) is available in Runtime 2.0.

Runtime 2.0 limitations (as of Feb 2026)

- Delta MERGE not yet available (blocks `Merge` and `Historic` strategies)
- Environment-based library management not supported (use `%%configure` with `spark.jars` instead)
- No starter pools (2-5 min session startup)
- No V-Order, autocompaction, or optimize write
- No R language support
- No Data Science integrations or Copilot
- No connectors (Kusto, SQL Analytics, Cosmos DB, MySQL)

JAR installation workaround for Runtime 2.0

Upload the JAR to a Lakehouse, then use inline configuration:

```
%%configure -f
{
  "conf": {
    "spark.jars": "abfss://<lakehouse>.dfs.fabric.microsoft.com/<path>/datalakefoundation.jar"
  }
}
```

Route B: Runtime 1.3 (if production needed before Runtime 2.0 GA)

Requires significant backporting effort:

- 1. Cross-compile to Scala 2.12** — Add `"2.12.17"` to `build.sbt`, fix any 2.13-only syntax.
- 2. Downgrade Spark/Delta API usage from 4.0 to 3.x:**
 - `DeltaMergeBuilder.execute()` returns `Unit` in Delta 3.x (not `DataFrame`) — rewrite metrics extraction in `Merge.scala:124-127` and `Historic.scala:106`.
 - `VARIANT` data type does not exist — replace with `STRING` in `TableAppender.scala:146`.
 - `parse_json()` function does not exist — replace with `from_json()` or store as plain string.
- 3. Verify `whenNotMatchedBySource`** — Available since Delta 2.4, should work on Runtime 1.3's Delta 3.2.

Codebase Portability Assessment

The codebase is well-architected for cross-platform use. Only two Databricks-specific code references exist:

Location	What	Impact
Unity Catalog config checks	Databricks Unity Catalog config checks	Falls back gracefully; non-blocking

Location	What	Impact
Databricks logger suppression	Suppresses Databricks-specific loggers	Harmless no-op on Fabric

Everything else (paths, Delta operations, metadata handling, processing strategies) uses standard Spark/Delta APIs and is metadata-driven.

Fabric Lakehouse Storage vs Databricks

Feature	Microsoft Fabric	Databricks
Storage layer	OneLake (ADLS Gen2)	DBFS / Unity Catalog
Table format	Delta Lake	Delta Lake
Catalog	Fabric workspace hierarchy	Unity Catalog
Path protocol	abfss://	dbfs://, abfss://, s3://
Path pattern	abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<lakehouse>/Tables/<table>	varies

The library's `PathLocation` mode maps to `abfss://` OneLake paths. The `TableLocation` mode maps to Fabric Lakehouse SQL endpoint table references. Both are configurable via metadata (`root_folder` environment setting).

Recommended Timeline

- 1. Now:** Test the JAR on Runtime 2.0 preview using the `Full` strategy only. Upload JAR to OneLake and use `%%configure` to load it.
- 2. When Runtime 2.0 gets MERGE support:** Test `Merge` and `Historic` strategies end-to-end.
- 3. When Runtime 2.0 reaches GA** (expected mid-late 2026): Deploy for production workloads with Environment-based library management.

References

- <https://learn.microsoft.com/en-us/fabric/data-engineering/runtime>
- <https://learn.microsoft.com/en-us/fabric/data-engineering/runtime-2-0>
- <https://learn.microsoft.com/en-us/fabric/data-engineering/runtime-1-3>
- <https://learn.microsoft.com/en-us/fabric/data-engineering/lifecycle>
- <https://learn.microsoft.com/en-us/fabric/data-engineering/library-management>
- <https://learn.microsoft.com/en-us/fabric/data-engineering/environment-manage-library>
- <https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-and-delta-tables>